

Reporte de práctica: API REST y pruebas automatizadas

Estudiante: Nombre del estudiante

Asignatura: Gestión y DevOps

Docente: Nombre del docente

2 de octubre de 2026

Resumen

En esta práctica se implementó y verificó una API REST para administrar elementos. El servicio está desarrollado con Node.js y Express, utiliza SQLite para la persistencia de datos y se empaqueta mediante Docker. Se configuraron pruebas automatizadas con Jest y Supertest para validar operaciones HTTP, validaciones de entrada y respuestas ante recursos inexistentes. La ejecución final de la suite completó satisfactoriamente las diez pruebas.

1. Objetivos

- Implementar una API HTTP para crear, consultar, actualizar y eliminar elementos.
- Persistir los registros en una base de datos SQLite.
- Verificar los casos exitosos y los errores esperados mediante pruebas automatizadas.
- Describir la configuración de ejecución local y en contenedor.

2. Tecnologías utilizadas

- **Node.js 18:** entorno de ejecución de JavaScript.
- **Express 5:** definición de rutas HTTP.
- **SQLite:** almacenamiento de elementos.
- **Jest y Supertest:** ejecución de pruebas y solicitudes HTTP.
- **Docker:** construcción y ejecución del servicio en un contenedor basado en Alpine.

3. Descripción de la solución

La API administra recursos `items`, cada uno con un identificador, un nombre y una descripción. La base de datos se crea o inicializa al iniciar el servidor. El servidor HTTP escucha en el puerto 80; además, el proyecto contiene un servidor TCP en el puerto 6061.

Las rutas HTTP principales utilizadas en la práctica son:

Método y ruta	Operación
GET <code>/items</code>	Consultar todos los elementos
GET <code>/items/:id</code>	Consultar un elemento por identificador
POST <code>/items</code>	Crear un elemento
PUT <code>/items/:id</code>	Actualizar uno o más campos
DELETE <code>/items/:id</code>	Eliminar un elemento

Las solicitudes de creación requieren `name` y `description`. La actualización permite enviar uno de esos campos, pero rechaza un cuerpo vacío o valores de texto vacíos. Si se consulta, actualiza o elimina un identificador inexistente, la API responde con estado HTTP 404.

4. Pruebas automatizadas

Las pruebas se encuentran en `api.test.js`. Supertest envía solicitudes al servidor local, y Jest comprueba los códigos de respuesta y la estructura básica de los datos. La URL de destino predeterminada es <http://localhost:80>; puede cambiarse mediante la variable de entorno `API_URL`.

N.º	Método	Escenario	Resultado
1	GET	Consulta de la colección <code>/items</code> .	Aprobada
2	POST	Creación de un elemento válido.	Aprobada
3	POST	Rechazo de un cuerpo vacío.	Aprobada
4	GET	Consulta del elemento recién creado.	Aprobada
5	GET	Respuesta 404 para un identificador inexistente.	Aprobada
6	PUT	Actualización del elemento.	Aprobada
7	PUT	Rechazo de un nombre vacío.	Aprobada
8	DELETE	Eliminación del elemento creado.	Aprobada
9	DELETE	Respuesta 404 al eliminarlo nuevamente.	Aprobada
10	GET	Respuesta 404 para una ruta inexistente.	Aprobada

Resultado de ejecución: 1 suite aprobada, 10 pruebas aprobadas y 0 fallidas.

4.1. Ejecución local

En una terminal, se inicia el servidor:

```
npm start
```

En otra terminal, desde la raíz del proyecto, se ejecutan las pruebas:

```
npm test
```

El servidor debe permanecer activo mientras se ejecuta la suite.

5. Contenedorización

El `Dockerfile` usa la imagen `node:18-alpine`, copia los manifiestos de `npm`, instala dependencias, copia el proyecto y expone los puertos 80 y 6061. La imagen inicia el servicio con `node server.js`.

Comandos para construir y ejecutar la imagen:

```
docker build -t api-items .  
docker run --rm -p 80:80 -p 6061:6061 api-items
```

Nota: el `Dockerfile` no configura un volumen para `SQLite`. Por lo tanto, para conservar la base de datos al eliminar el contenedor se debe configurar almacenamiento persistente.

6. Conclusiones

La práctica permitió integrar una API REST con persistencia `SQLite` y pruebas automatizadas. La suite valida operaciones `CRUD`, reglas básicas de entrada y respuestas de error. La ejecución satisfactoria de las diez pruebas confirma que los comportamientos cubiertos están implementados conforme a las expectativas definidas en el proyecto. Docker proporciona una forma reproducible de empaquetar el servicio, mientras que la persistencia requiere configurar un volumen cuando los datos deban sobrevivir al ciclo de vida del contenedor.